Lab 13 — Services en Ingress: blootstellen

Theorie — waarom het IP van een Pod nergens toe dient, hoe een stabiele naam aan veranderende Pods gekoppeld wordt, en langs welke deuren het verkeer het cluster binnenkomt: ClusterIP, NodePort, LoadBalancer, Ingress.

Doelstellingen

- Begrijpen wat een **Service** aan Pods toevoegt: een virtueel IP, een DNS-naam, een verdeling.
- `ClusterIP`, `NodePort` en `LoadBalancer` uit elkaar houden en weten welk je gebruikt.
- De **EndpointSlices** lezen en het verband met de readiness begrijpen.
- De frontend blootstellen via een **Ingress** en de rol van de Ingress controller (Traefik) begrijpen.
- In dat alles de DNS en de reverse proxy van de labs 07 en 09 terugvinden.

---

1. Het probleem: IP's die voortdurend veranderen

In lab 12 verving elke rollout de Pods, en elke nieuwe Pod kreeg een nieuw IP `10.42.0.x`. Een frontend die het adres van een API-Pod vast ingeschreven had, was bij de eerste update gevallen. Erger nog: met drie replica's, welk aanroepen? In het praktijklab faalt nginx, gestart vóór enig Service, onmiddellijk: `host not found in upstream "api"` — niets in het cluster draagt die naam.

Het **Service** lost beide problemen op: het levert een stabiele **naam** en een stabiel **IP** (het cluster-IP, `10.43.x.x` bij k3s, nooit opnieuw toegekend zolang het Service bestaat) vóór een **verzameling Pods** gekozen op label, en verdeelt de verbindingen erover. De Pods komen en gaan; het Service blijft.

> **→ DOCKER-HERHALING**
> In lab 07 gaf het user-defined netwerk elke **container** een DNS-naam; in lab 09 deed Compose dat voor elke **dienst**, en gaf `--scale api=2` twee adressen aan dezelfde naam. Het Kubernetes-Service is de voldragen versie van dat idee: één naam, één virtueel IP, een lijst echte adressen die het cluster bijhoudt, en een verdeling door de node in plaats van door de client.

2. Anatomie van een Service

```
apiVersion: v1
kind: Service
metadata:
  name: api                 # de DNS-naam
spec:
  selector:
    app: api                # de beoogde Pods, op label
  ports:
    - port: 8080            # poort van het Service (die de clients aanroepen)
      targetPort: 8080      # poort van de container
```

Achter dat bestand schuilen drie mechanismen. **De DNS**: CoreDNS maakt de vermelding `api.lab13.svc.cluster.local` → cluster-IP; vanuit dezelfde namespace volstaat `api` dankzij de `search` in de `/etc/resolv.conf` van de Pods. **De EndpointSlices**: een controller inventariseert

voortdurend de IP's van de Pods die het label dragen **en** `Ready` zijn; `kubectl get endpointslices` toont ze, en dat is de echte lijst achter het virtuele IP. **kube-proxy**: op elke node programmeert hij regels (iptables of nftables) die elke verbinding naar het cluster-IP herschrijven naar een verbinding naar een van de endpoints, willekeurig.

> **→ NETWERK**
> Het cluster-IP hangt aan **geen enkele interface**: geen proces luistert erop, `ping` antwoordt niet. Het is een louter virtueel adres dat de **NAT**-regels van kube-proxy ter plekke omzetten in een Pod-adres, op de node waar het pakket vertrekt. De verdeling gebeurt dus per verbinding, op laag 4 (TCP), zonder begrip van HTTP-verzoeken — een client die zijn verbinding open houdt, blijft op dezelfde Pod.

Twee praktische gevolgen. Een Pod die **niet klaar** is (readiness faalt, lab 12) verdwijnt uit de EndpointSlices: het Service stuurt hem geen verkeer meer zonder dat hij gedood wordt — dat is de hele zin van de readiness. En een Service **zonder selector** heeft geen endpoints: nuttig om een database buiten het cluster te benoemen (met een handgeschreven EndpointSlice), een valkuil wanneer het label verkeerd gespeld is (`kubectl get endpointslices` leeg = eerste reflex).

3. De types Service: langs waar komt het verkeer binnen

Type	Bereikbaar vanuit	Hoe	Wanneer
ClusterIP (standaard)	binnen het cluster	virtueel IP + DNS	elke interne dienst: `api`, `db`
NodePort	buiten, via een poort van **elke node** (30000-32767)	`IP-van-node:30080` → Service	test, lab, toegang zonder load balancer
LoadBalancer	buiten, via een IP dat de infrastructuur levert	een externe load balancer → NodePort → Service	publieke blootstelling in de cloud
ExternalName	binnen	een DNS-alias naar een externe naam	een dienst buiten het cluster benoemen

Elk type **omvat** het vorige: een NodePort heeft ook een ClusterIP, een LoadBalancer ook een NodePort. De NodePort toont een beperking: je moet het adres van een node kennen, en de poort is vreemd. De LoadBalancer verhelpt dat door de infrastructuur een "net" adres te vragen — in de cloud is dat een gefactureerde load balancer; op een kaal cluster blijft het `EXTERNAL-IP` voor altijd `<pending>`, bij gebrek aan leverancier.

> **K3S**
> k3s levert **ServiceLB** (vroeger Klipper): voor elk Service `LoadBalancer` start het een Pod `svclb-<naam>` in `kube-system` die de gevraagde poort **op de node** opent en doorgeeft; het `EXTERNAL-IP` wordt het IP van de node. Twee Services kunnen dus niet dezelfde poort vragen (het tweede blijft `<pending>`), en daarom bezet Traefik al de poorten 80 en 443. Op EKS/AKS/GKE levert hetzelfde manifest een echte load balancer met een publiek IP: de YAML verandert niet, de factuur wel.

> **→ WINDOWS / WSL**
> Een NodePort of een ServiceLB-poort bereik je vanuit Windows op `http://<ip-van-de-distributie>:30080` (`hostname -I` in WSL), en meestal op `http://localhost:30080` dankzij de WSL-forwarding — voor de poorten die k3s opent, dat als root draait, geldt de beperking op poorten < 1024 uit lab 07 niet: Traefik luistert wel degelijk op **80**. `kubectl port-forward` blijft de veiligste weg voor een snelle test: het hangt van geen enkel Service-type af.

4. De Ingress: één deur voor alle HTTP

Elke frontend via een LoadBalancer blootstellen kost een adres (en een load balancer) per applicatie. De **Ingress** bundelt: één enkel HTTP/HTTPS-toegangspunt, en **regels** die per hostnaam (`app.example.com`) en per pad (`/api/`) naar Services routeren. Het Ingress-object is maar een **beschrijving**; het is een **Ingress controller** — een reverse proxy die in het cluster draait en die objecten leest — die het werk doet: Traefik bij k3s, ingress-nginx, HAProxy, of de applicatieve load balancer van de cloud.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

Het verkeer volgt dan: browser → poort 80 van de node (ServiceLB van Traefik) → Traefik-Pod → Service `web` → nginx-Pod → (voor `/api/`) Service `api` → API-Pod. Twee reverse proxies na elkaar, Traefik en dan nginx: dat is gebruikelijk, en daarom laten veel teams `/api/` rechtstreeks door de Ingress naar het Service `api` routeren, en halen ze de proxy uit nginx.

> **→ HTTP**
> De Ingress werkt op laag 7: hij leest de header `Host` en het pad van elk verzoek, kan **TLS** beëindigen (certificaten, vaak geautomatiseerd door cert-manager), paden herschrijven, headers toevoegen (`X-Forwarded-For`, waardevol na lab 07), het debiet beperken. Een Service ziet alleen TCP-verbindingen. De opvolger van de Ingress, de **Gateway API**, veralgemeent die regels; Traefik ondersteunt ze, maar de Ingress blijft de standaard die overal uitgerold is.

5. De diagnose in vier vragen

Wanneer "het niet antwoordt", de keten teruglopen: (1) **zijn de Pods klaar?** `kubectl get pods` — `0/1` = readiness; (2) **heeft het Service endpoints?** `kubectl get endpointslices` leeg = verkeerde selector of Pods niet klaar; (3) **resolvet de naam?** `kubectl exec … -- wget -qO- http://api:8080/` vanuit een Pod van dezelfde namespace; (4) **is de toegangsdeur de juiste?** `kubectl get svc` voor type en poort, `kubectl get ingress` voor adres en regels, `kubectl describe ingress` voor de backends. Een `503` van Traefik betekent bijna altijd "het beoogde Service heeft geen enkel klaar endpoint".

6. In het bedrijf

- **Eén ClusterIP-Service per Deployment**, met dezelfde naam, in hetzelfde manifest: `api` en `db` worden nooit buiten het cluster blootgesteld.

- **Eén enkele Ingress controller** voor het hele cluster, beheerd door het platformteam; elke applicatie declareert haar Ingress-regels (of Gateway) met een hostnaam en een TLS-certificaat.
- **De frontend roept de API aan via een relatief pad** (`/api/`), gerouteerd door de Ingress of door nginx: dezelfde origin, geen CORS (lab 07), en de URL verandert niet van omgeving tot omgeving.
- **NodePorts** blijven op het werkstation en in het lab; de `LoadBalancer` is voorbehouden voor de zeldzame niet-HTTP TCP-diensten (gerepliceerde databases, brokers), en wordt gefactureerd.

Onthouden

- Een Service = een DNS-naam + een stabiel cluster-IP + een lijst endpoints bijgehouden op label en readiness + een verdeling door kube-proxy.
- `api` resolvet in de namespace, `api.lab13.svc.cluster.local` overal; zonder Service sterft nginx op `host not found in upstream`.
- ClusterIP voor intern, NodePort om te testen, LoadBalancer voor een extern IP (ServiceLB bij k3s, cloud-load-balancer elders), Ingress voor alle HTTP.
- Een Pod die niet klaar is, verdwijnt uit de EndpointSlices; een verkeerde selector geeft een Service zonder endpoints en een `503`.
- De Ingress beschrijft regels; de Ingress controller (Traefik) past ze toe op poort 80/443 van het cluster.
- Diagnose: Pods klaar → endpoints → DNS → toegangsdeur.

Woordenschat

Service: stabiele naam en IP vóór Pods geselecteerd op label. — **ClusterIP**: virtueel IP binnen het cluster. — **NodePort**: poort geopend op elke node (30000-32767). — **LoadBalancer**: extern IP geleverd door de infrastructuur (ServiceLB bij k3s). — **EndpointSlice**: lijst van IP's van klare Pods achter een Service. — **kube-proxy**: NAT-regels per node die het Service realiseren. — **CoreDNS**: DNS van het cluster (`<svc>.<ns>.svc.cluster.local`). — **Ingress**: HTTP-routeringsregels per host en pad. — **Ingress controller**: reverse proxy die de Ingresses toepast (Traefik). — **Gateway API**: opvolger van de Ingress.